iHelp — Basic Security

1. The Security Model in One Paragraph

The database is the only authority. The UI hides what you cannot do, the Server Actions re-check it for friendly errors — but both are convenience layers. The layer that decides is PostgreSQL: Row Level Security policies for row access, table grants for verb access, CHECK/unique constraints for value and cross-row rules, and eleven SECURITY DEFINER functions (each with in-body permission checks) for atomic state transitions. A crafted HTTP request carrying a valid user JWT — bypassing our UI and server entirely — hits exactly the same wall, and the integration suite proves it by attempting every forbidden action that way (tests P1–P15).

2. Authentication

- **Supabase Auth, email + password.** Three Server Actions in `actions/auth.ts` — `signUp` (→ `/profile` onboarding), `signIn` (→ `/requests`), `signOut` (clears the cookie → `/login`). Failed login returns a generic "email or password incorrect" (no hint which was wrong).
- **Sessions are cookie-based via `@supabase/ssr`.** The middleware (`proxy.ts` → `updateSession`) calls `getUser()` on every matched request, which refreshes an expired access token and re-writes the cookie; every Server Component / Server Action builds a per-request client from those cookies — so the user's **JWT** rides every DB and storage call, and Postgres sees `auth.uid()` = that user (this is the hinge of the RLS model).
- **Cookie flags** (the library's hardened defaults, which we rely on rather than hand-set): **HttpOnly** — JS cannot read the token, so a hypothetical XSS still can't steal the session; **Secure** — HTTPS-only in production; **SameSite=Lax** — the browser won't attach the cookie to cross-site subrequests (CSRF defense, see §7). The token is a short-lived signed JWT (`sub` = user id, `role`, `exp`) plus a refresh token used transparently by the middleware.
- Passwords: minimum 8 characters (zod + form attribute); **hashing is bcrypt inside Supabase (GoTrue)** — we never store, compare, or log a raw password. Token rotation and brute-force throttling are likewise Supabase Auth's responsibility, not custom code — deliberately, because hand-rolled auth is the classic way to get this wrong.
- **Email confirmation is OFF for the MVP** (demo-day reliability, no SMTP dependency). Consequence: anyone can register with an email they don't own. This is listed as risk R1 in §9 — it does not weaken row security (identity in the DB is the UUID, not the email), but it allows email squatting and is the first switch to flip after the course demo.

3. Authorization

Authorization is layered, and only the last layer is trusted (architecture §9):

1. **Middleware** — session refresh + signed-out redirect. No authorization.
2. **UI** — hides actions per role/state. Usability only.

3. **Server Actions** — zod validation + fast pre-checks. Friendliness only.
4. **PostgreSQL** — the authority: - **RLS policies** on all seven tables (every policy with its justification: technical design §2). No table has an `anon` policy of any kind. - **Eleven SECURITY DEFINER RPCs** own every state transition (assign, dual-complete, cancel, rate, review, revoke, hide, mark-paid, create request, contact reveal, set final price). Each starts with a permission check; row locks make the transitions atomic (the withdraw-vs-assign race test proves the rollback). `set_final_price` is the model example of the in-body checks: only the *selected helper* may call it, only for an `after_job` offer, only once (a set `final_price` rejects a second call). - **Column-guard triggers** close what row-level policies cannot express: nobody PATCHes `status`, `is_paid`, verification flags, or `is_admin` — tested by attempting exactly that (P9, P10). - **Explicit least-privilege grants**: `authenticated` has **no INSERT** on `help_requests` / `request_photos` / `ratings` (writes exist only inside definer RPCs) and **no DELETE anywhere**; `anon` has no table access. Every verb each role holds is pinned explicitly in the schema — the app never relies on platform default privileges for security. - **Admin is data, not code**: `is_admin` lives in `profiles_private`, set only by direct SQL, checked by a definer helper inside policies and RPCs. Admin *capabilities* are exactly the three admin RPCs (`review_application`, `revoke_verification`, `set_request_hidden`) — there is no blanket admin table access.

4. What Requires Being Signed In

Anonymous visitors	Signed-in (unverified)	Identity-verified	Admin
Landing, login/signup, **emergency page** (static, no data)	Browse requests/helpers, edit own profile, apply for verification	+ Post/edit/cancel own requests, offer, assign, complete, rate, mark paid, see assigned counterpart's contact	+ Review applications, hide/unhide requests, revoke verifications

Anonymous callers get *nothing* from the data layer — no policies target `anon`, no grants exist for it, and test P15 verifies zero rows/denials on every surface including the ratings view.

5. Preventing Access to Other Users' Data

The cases that matter, and their mechanisms (each has a denial test):

Data	Protection
Phone + home coordinates + admin flag	Live in `profiles_private`, a separate table with own-row-only policies — because RLS is row-level and the public profile row must be broadly readable. Cross-user phone access exists only through `get_counterpart_contact` (parties of an assigned request, post-assignment)
Competing offers (sealed bids)	`offers` readable only by the offer's owner and the request's owner (P5)
Requests in later states	Feed rule hides assigned/cancelled/hidden rows from non-parties (P6); owner, selected helper, and admins retain access
Who-rated-whom linkage	Base `ratings` table is party-scoped; third parties read through the `helper_ratings` view, which exposes stars/note but has no `rater_id` / `request_id` columns (D7)
Verification documents (ID photos, certificates)	Private bucket; storage policies allow the applicant's own folder + admins only; delivery via short-lived signed URLs
Row existence itself	RPCs follow the error-ordering rule — a non-party probing any id gets `not_found` before any state information leaks (P7/P8/P12); RLS-invisible rows 404 identically to missing ones

6. Input Validation

Three rings, outermost to innermost (each mirrors the same bounds):

1. **Client**: native HTML constraint attributes (`required`, `minLength`, `min/max`, `type`) for instant feedback; file type/size checked before upload (≤ 5 MB, JPEG/PNG/WebP, ≤ 5 files).
2. **Server Action**: the authoritative `zod` parse — every field bounded, enums closed, Israeli phone regex, offer-price bounds, and the two edge cases baked into tests (absent-field `null`s, coordinate coercion to (0,0)).
3. **Database**: CHECK constraints (lengths, ranges, category set, offer-price bounds, phone format), enum types (invalid state = type error), unique indexes, and RPC-body validation (photo paths must be real objects in the caller's own folder — X6).

The three named web-security threats

- **XSS (cross-site scripting)** — React escapes all rendered content by default and the codebase contains **no** `dangerouslySetInnerHTML`, so stored-XSS via titles/messages/notes is inert text at render, never executed. Auth cookies are **HttpOnly**, so even a hypothetical XSS can't read the session token.
- **SQL injection** — **structurally impossible in our code**: we write no dynamic SQL and no string concatenation into queries. All access is either the Supabase query builder (parameterised) or `SECURITY DEFINER` RPCs called with **typed, bound arguments** (`supabase.rpc(name, {arg: value})`); inside the functions `search_path` is pinned. Postgres receives values as parameters, never as concatenated SQL text.
- **CSRF (cross-site request forgery)** — writes are Next.js **Server Actions**: same-origin POSTs with a framework-encrypted action identifier, so a third-party page can't forge a call, and there is no

REST endpoint to target. Session cookies are **SameSite=Lax**, so the browser won't attach them to cross-site subrequests. (See §7.)

7. Protecting API Calls

- **We expose no API of our own.** Mutations are Next.js Server Actions — same-origin POSTs with encrypted action identifiers; there are no REST route handlers to enumerate or forget to protect.
- **The reachable API surface is Supabase's PostgREST/storage/auth endpoints**, and that surface is exactly what the RLS policies + grants + RPC checks authorize — this is the audited surface of §3, deliberately identical for our UI and for a curl attacker.
- **The service-role key appears in zero lines of application code** (grep-able claim). It exists only for local tooling (seed script, test harness admin setup) and is never configured on Vercel. Consequence: compromising the deployed environment's variables yields only the anon key — which grants nothing beyond what any signed-up user already has.
- Delivery from the two **private** buckets (`request-photos`, `verification-docs`) uses **1-hour signed URLs** created server-side under the caller's own storage permissions — links expire instead of living in HTML forever. The third bucket, `avatars`, is **public by design**: avatars render in `<img>` across the app and are non-sensitive public-identity data like `display_name`, so a public bucket avoids per-render signed URLs. Writes are still policy-scoped to the owner's own folder, with a 2 MB size limit and a JPEG/PNG/WebP MIME allowlist.

8. Secrets and Environment Variables (required list)

Variable	Where	Sensitivity
`NEXT_PUBLIC_SUPABASE_URL`	Vercel + local	Public by design
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Vercel + local	Public by design — RLS is the authority (§1); exposure grants nothing beyond any signed-up user
`SUPABASE_SERVICE_ROLE_KEY`	**Local only** (seed/tests)	Critical — bypasses RLS; never on Vercel, never read by app code
`SEED_PASSWORD`	Local only (optional)	Demo accounts' password; if unset the seed generates a random one and prints it once

No secret is committed to the repository (`.env*` is gitignored; the example file contains placeholders). Supabase-side secrets (JWT signing secret, DB password) never leave Supabase's dashboard. The demo seed password is supplied through the `SEED_PASSWORD` environment variable (or randomly generated and printed once by the seed script), never embedded in code — see risk R9.

9. Remaining Risks and What We Would Improve Next

Honest list, ordered by how soon each should be addressed:

#	Risk	Current bound	Improvement
R1	Email confirmation off → register with someone else's email	Identity = UUID; email squatting only	Flip Supabase's confirmation switch + SMTP (one config change; the auth callback route it needs is documented)
R2	No app-level rate limiting → RPC/auth hammering	Supabase Auth's built-in limits; DB constraints cap damage (one active offer, one open application, 5×5MB photos)	Edge-middleware token bucket per user; stricter auth rate config
R3	Identity proofing is manual review, phone unvalidated	Human gate + optional ID photo; documented openly (spec §4.1)	SMS OTP on the phone at application time (accepts the external-service trade-off post-MVP)
R4	Request coordinates queryable by any signed-in user	UI never ships them; accepted in spec §9.3	Round coordinates at rest, or server-side-only distance via RPC
R5	Photos of hidden requests remain fetchable by known path	Unguessable UUID paths; hide = feed removal, not secrecy	Storage policy joined to the parent request's visibility
R6	ID-photo retention indefinite while account is verified	Private bucket, admin-only read	Retention job: delete document objects N days after a decision
R7	Phone remains visible to the counterpart after completion	By design (coordination); parties only	Time-box the contact reveal (e.g., revoke N days after `rated`)
R8	Single human admin = availability and abuse single-point	Full audit trail (`verification_applications`, `decided_by`)	Second admin + four-eyes revocation; audit log surface
R9	Demo seed accounts on the presentation instance	Env-supplied password, printed once	Rotate/delete demo users right after the demo
R10	Dependency drift (Next/Supabase/zod majors move fast)	Lockfile pins; CI-less repo relies on local runs	Dependabot + the test suite as the upgrade gate

One class of attack is closed by design rather than left on this list: the offer INSERT policy pins `final_price is null`, so a direct insert can never pre-set the agreed amount — the after-job price can only be set through the guarded `set_final_price` RPC after completion.

None of these breaks the core guarantee of §1 — they are erosion risks at the edges, each with a bounded blast radius and a named, additive fix.